

Foundations of Tokens, Embeddings, and Semantic Representation

Generative AI and LLMs

Prof G C Nandi

Full marks-100

LLMs and Numbers: Large Language Models (LLMs), like all computers, can only understand numbers. To represent words in a structured way, we convert them into numbers using **tokenization** and **embeddings**.

For now, please focus on learning the following key concepts:

1. Tokenization: Think of language as building with Lego blocks. A *token* is the smallest block we use after breaking down text. Sometimes a token is a whole word like “*cat*” or “*the*”, and sometimes it’s just a piece of a word like “*run*” + “*ning*”. Since computers only understand numbers, each token is given a unique ID (like a number in a vocabulary list). That’s how we “talk” to large language models—by turning words into tokens and then into numbers.

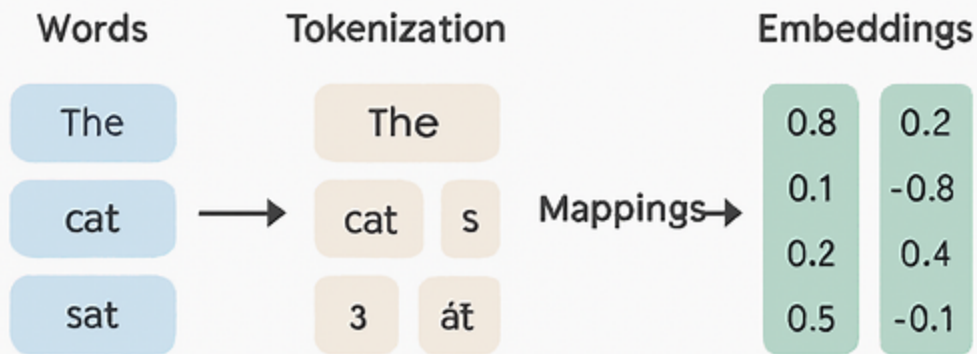
LLMs don’t always use whole words as tokens. Instead, they often break words into smaller parts using methods like **Byte Pair Encoding (BPE)** or **WordPiece**. For example, the word “*unhappiness*” might be split into “*un*”, “*happy*”, and “*ness*”. This way, the model can handle rare words, spelling mistakes, and even multiple languages more easily.

2. Embedding Vectors (Dense Representations): Imagine each word as a point on a map. A onehot vector is like giving every word its own isolated spot with no connection to others. In contrast, *embeddings* place words in a shared space where their positions reflect meaning. During training, the model learns these positions so that related words end up close together—for example, “*man*” near “*woman*”, and “*king*” near “*queen*”. Because of this, embeddings can capture relationships and analogies, such as: *Man* → *King*, *Woman* → ? → *Queen*.

3. Positional Encoding: Word embeddings tell us the meaning of words, but they don’t tell us the order in which words appear. To fix this, Transformers add extra information called *positional encoding*. Think of it like giving each word a “position tag” so the model knows whether a word comes first, second, or later in a sentence. One common way to do this is by using patterns called sinusoidal encodings. We’ll look at how these work more closely when we study the Transformer architecture later in class.

LLMs and Numbers

Computers don't really "understand" words -they only understand numbers. So, when we want to teach a computer language, we have to translate words into numbers.



The picture shows the flow from **Words** → **Tokens** → **Numbers** → **Embeddings**.

(further study our text book- Hands-On LLMs, Chapter-2)

Assignments#1:

In industry, embedding for recommendation system is widely used.

Use word2vec algorithm to embed songs and use human made music playlist and then use these embeddings to recommend similar songs that often appear together in the playlist. Treat each song as a word or token and playlist like a sentence.

[hints: Use Cornell University dataset (<https://oreil.ly/A-AK6>), train a Song Embedding Model and use those embeddings to find similar songs. Ref Chapter -2 of our text book].

Assignment#2

Q1. Tokenization & Meaning (10 Marks)

Explain the following in your own words:

- a) Why do LLMs not operate directly on words?
- b) How subword tokenization helps with:
 - rare words
 - morphology
 - open vocabulary
- c) Give **one example** where word-level tokenization fails but subword tokenization works well.

Q2. Embedding Space Intuition (10 Marks)

Consider the words:

king, queen, man, woman, doctor, nurse

- a) What does it mean to say these words lie in a “vector space”?
- b) Why is **cosine similarity** preferred over Euclidean distance in embedding spaces?
- c) Can embeddings capture **bias**? Briefly explain.

Q3. Semantic vs Lexical Similarity (10 Marks)

Explain the difference using the following sentence pairs:

1. “*How to bake a cake?*”
“*Steps for making a cake*”
2. “*Apple released a new phone*”
“*Apple is a healthy fruit*”

Which pairs will embedding similarity score higher, and why?

Hands-On Embeddings (50 Marks)

Students may use HuggingFace (Laptop friendly- no GPU required)

Q4. Embedding Generation & Similarity (20 Marks)

Task:

1. Choose **any pre-trained embedding model**
2. Compute embeddings for the following sentences:

S1: Robots can assist humans in daily tasks

S2: Machines help people in everyday activities

S3: The stock market crashed yesterday

3. Compute **pairwise cosine similarity**
4. Comment on the results

Expected Deliverables:

- Code
- Similarity matrix
- 5–6 lines interpretation

Q5. Mini Semantic Search Engine (25 Marks)

Task:

Build a **tiny semantic search system** using embeddings.

Corpus (minimum 8 documents):

- Choose from any one domain:
 - Healthcare
 - Robotics
 - Education
 - News

Steps:

1. Convert documents to embeddings
2. Accept a user query
3. Retrieve **top-3 most similar documents**
4. Display similarity scores

Q6. Limitations of Embeddings (5 Marks)

Answer briefly:

- a) Can embeddings fully capture **context**?
- b) Why might embeddings fail for **numerical reasoning**?
- c) Why do embeddings struggle with **negation**?